AI Stalemate



THE LEAD DEVELOPER FIELD GUIDE

Your team is being asked to build differently. Prepare your recommendation.

Nic Payne

For you, the lead developer

OPEN YOUR LEAD DEVELOPER LAB >

HOPE IS REBELLIOUS.

WORKING EDITION / REBEL-LEADER.COM

Your PM wants the team to build differently.

They keep bringing up AI. You keep seeing work that a quick demo leaves out. Now you're being asked to change how the team builds and ships, and you'll be responsible for what happens afterward.

You have reasons for being concerned

Perhaps the team is still learning how to test, debug, and review its own work. Perhaps the new app needs restricted data that has no access setup in your Azure environment. Faster code generation doesn't settle either question. Start with what you can explain from the work in front of you.

There may be something your PM sees too

They may be hearing about people waiting, work being done by hand, or an opportunity the business keeps missing. Ask for that context. You can take the opportunity seriously while disagreeing with the proposed way of delivering it.

What this tool helps you do

Get clear on the change being requested, explain your position, understand what matters to your PM, and recommend a next move. That might be a smaller change, specialist help, a normal engineering spike, or a decision not to proceed. You do not have to agree to adoption to have a useful conversation.

Start here: "When you say we should use AI, what do you want the team to do differently?"

Reading and reflection: see the companion at the end of this Guide >

Why does this matter so much to my PM?

Before you answer the proposal, find out what is behind it. You may discover a problem you can help solve in a way neither of you has considered.

Ask about the work people are waiting for

“Who needs this? What are they doing today? What would change if we delivered it?” Ask what makes the date matter and what has already been promised. If a benefit is an estimate, keep it an estimate. Your PM should be able to explain the need without defending a particular AI product.

Find out where the pressure is coming from

“Is this about this app, our delivery pace, or changing how the whole team works?” Those are different requests. An executive’s demo may have made a possibility feel immediate. Ask what the demo actually showed and what people now expect.

Bring your view of delivery too

Where does work really wait: unclear requirements, access approvals, coding, review, testing, or deployment? Own the problems engineering can improve. Ask your PM to help with the decisions and dependencies that engineering cannot resolve alone.

Try: “I can see why this matters. Help me understand what you need to change, then I’ll explain where I think this approach gets us into trouble.”

Reading and reflection: see the companion at the end of this Guide >

Start with what your PM needs.

You can be curious about the request and clear about your responsibility at the same time.

What is happening?

“Who is waiting for this? What is difficult for them today?” Ask for examples you have not seen.

What has been promised?

“What did we tell people we could deliver, and by when?” Find out whether a demo has become a commitment.

Which change matters?

“Do we need this app sooner, or are we changing how the team develops?” Separate the immediate need from the larger ambition.

Where could you help?

“Here’s where I see work getting stuck. Could we look at those delays together?” Include the parts engineering can improve.

Listen for context that could change your recommendation. You are not committing to a solution by asking about the problem.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

“Use AI” can mean very different things.

Separate the way you build software from what the software does. An app can be built with a coding assistant and contain no AI at all. A useful AI feature does not require autonomous development.

Help with a task, or hand over a workflow?

An engineer asking for help with a test is different from agents planning work, changing files, running tools, and opening changes across a codebase. Agree the scope, permissions, review points, and responsibility. The amount of supervision matters as much as the tool name.

Does the app itself need AI?

A restricted internal app may only need ordinary forms, rules, search, and an approved data connection. If it needs categorization, compare rules, an existing model, a trained classifier, and other options against real examples. Start with the job and the cost of getting it wrong.

Jev is one example of that distinction

TypeSafe describes Jev as a decision model for structured outputs such as categories and scores, rather than a conversational assistant. That makes it a different candidate from a coding agent or a chat interface. A valid category can still be the wrong category; evaluate errors, data handling, and fit before choosing it.

TypeSafe · Introducing System One models and Jev

Ask: “Are we choosing how to build this, what the app needs to do, or both?” Write the two decisions separately.

Reading and reflection: see the companion at the end of this Guide >

Get clear on your own position.

You do not need a polished argument. You need a clear account of what concerns you, what supports it, and what you would recommend. The first Lab step is private preparation.

Separate what you know from what you expect

“We have not defined who can access these records” is a known gap. “The team will be unable to review agent-written code” is a prediction unless you have seen that happen. Use examples from their work. A known requirement does not become optional because someone wants to move quickly.

Say what your concern changes

Explain the consequence and the next decision: “Without a permissions model, we cannot say who this app may expose the data to. We need the data owner and platform engineer involved before using real records.” Make the work and its owner visible.

Leave room for their answer

Notice if you have already decided your PM only cares about a shiny tool. Ask what they are trying to accomplish. Be equally clear about your own position: something you can support now, something that needs preparation, or something you recommend against. Curiosity does not require setting aside your judgment.

Prepare one sentence: “My concern is __. I’ve seen __. I recommend __. What am I missing?”

Reading and reflection: see the companion at the end of this Guide >

Can the team own what the agents produce?

If people are still learning engineering fundamentals, giving them a larger volume of unfamiliar code can create more work for the few people able to review it. Assess the task and the support available.

Look at demonstrated skills

Can the people doing this explain the code, recognize a bad assumption, write a useful test, debug a failure, and recover a release? Look at recent work together. A job title or years of experience alone cannot answer those questions. Avoid turning the conversation into a verdict on the team.

Protect the chance to learn

A small randomized study of developers learning an unfamiliar Python library found lower immediate quiz scores with AI assistance. It does not establish what will happen to your team over time. It does make “Did they finish?” an incomplete learning check. Ask people to explain and debug their work as well.

Match the change to the support

You might support one engineer using assistance on a small task with an experienced partner, while declining an autonomous workflow across several services. Name the review capacity, learning time, and fallback. If no one can confidently review the result, that is a staffing constraint to resolve.

Anthropic · How AI assistance impacts the formation of coding skills

Try: “I’m open to helping the team learn this. I’m not comfortable asking them to own code they can’t yet explain or debug. Here’s the support I think they need.”

Reading and reflection: see the companion at the end of this Guide >

Who is allowed to see this data?

An executive asks for a small app using a restricted dataset. The app, user groups, and data connection are not already set up in Azure. The interface may be quick to build; the access decisions still need owners.

Sign-in is only part of the answer

Who may read or change which records? Who approves access and removes it when someone leaves or changes roles? Microsoft's App Service documentation distinguishes authentication from finer-grained authorization. Having an Entra sign-in does not by itself establish the app's data permissions.

Check the system, not just the screen

Agree where permissions are enforced in the server or API. Test allowed and denied requests, including direct requests for someone else's records. Name the service identity, approved data connection, secrets handling, logging, and person responsible for support. Use the organization's architecture and security owners to settle the design.

Keep agent permissions separate

Who may use the app and what a coding agent may access are different questions. Decide whether the development tool can see data, credentials, repositories, or deployment environments. Restrict access to what the task needs. An isolated prototype with invented records does not prove production access is ready.

Microsoft · App Service authentication and authorization

OWASP · Authorization Cheat Sheet

Try: "We can explore the interface with made-up data. Before real people use real records, we need an agreed access model and someone accountable for checking it."

Reading and reflection: see the companion at the end of this Guide >

An app and a development workflow.

In the fictional example, a restricted-data app and agentic development arrive as one request. Pull them apart before deciding.

The app needs a foundation

Who can access which records? Who owns the policy, connection, support, and release? This work exists however the code is written.

The team needs a workable approach

What can people explain, debug, test, and review? What can an agent touch? Who has time to supervise and correct the work?

You can investigate separately

Ask the data owner and Platform about access. Consider an isolated development task with invented data only if it answers a useful question.

Keep the conclusions separate

A promising development spike does not approve the app's permissions. A sound access design does not prove the team can operate an agent workflow.

Write the missing work, its owner, and what would show it is ready. Do not promise a production app from a prototype.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

What do we actually need to find out?

Engineering teams already research tools and run spikes. Use those practices when they answer a real question. You do not need a new experiment to establish a requirement you already know.

Use the evidence you already have

Bring a recent review, incident, access requirement, or delivery bottleneck. Read the relevant documentation or ask a specialist before rebuilding what is already understood. Distinguish “we do not know yet” from “we know this needs to be done.”

Compare workable alternatives

Could you narrow the app, use an existing approved platform, automate one part with ordinary code, or change a process? Could assisted development help a bounded task even if agent orchestration is too much today? Describe what each option helps with and what work remains.

Choose the smallest useful next move

That may be a conversation with the data owner, a feasibility spike, time learning a tool together, or a decision to stop this use. GitHub’s guidance recommends simpler tasks when starting with coding agents and flags security-sensitive or broadly scoped work as potentially better kept with a developer.

GitHub · Best practices for using Copilot to work on tasks

Try: “We have two different unknowns. Let’s ask Platform about access first. Then we can decide whether a small coding spike would tell us anything useful.”

Reading and reflection: see the companion at the end of this Guide >

Explain the concern. Offer a recommendation.

Fictional example: your PM wants the team to use agents to deliver an internal app after an executive demo. The dataset needs restricted access, and the team is still learning to review and support its code.

Hear the request

PM: “People keep waiting on us for small apps. I’d like us to try building this differently.” Lead: “I’d like to improve that too. Is the priority getting this app to those users, or proving a new way of developing? What did you tell them we could do?”

Make the concern specific

Lead: “We don’t have the access setup for this dataset. And the team still needs help reviewing ordinary changes. I don’t think asking them to manage several agents is a responsible starting point. We’d be adding work they can’t yet check.”

Propose a next move without overpromising

Lead: “I recommend we get the data owner and Platform together first. In parallel, if we can free up an experienced reviewer, we could explore one small task with invented records. That would tell us about the workflow, not whether the app is ready for production.”

Leave room for disagreement

PM: “I can help with those people. I still want to understand whether we’re missing a faster option.” Lead: “Fair. Let’s compare an existing approved tool too. We haven’t agreed the spike yet. Can we come back Friday with the options and who is available?”

Afterward, write what was actually agreed, what is still your recommendation, and what you still see differently. The Lab’s Conversation Notes can help.

Reading and reflection: see the companion at the end of this Guide >

Make your position easy to understand.

Use these prompts to prepare. Choose the ones you need; this is a conversation, not a script to perform.

The concern

"I'm worried about __. Here's what we've seen, and here's what I'm still predicting." Give one concrete example.

The consequence

"That means we can't yet __." Explain what the concern changes about the proposed work.

The alternative

"I could support __, provided we have __." Offer a smaller change, a different solution, or preparation where it would help.

The question

"Would that address what you need? What am I missing?" Let the PM's answer affect the recommendation.

You can still disagree afterward. Record the difference fairly, name who can decide, and agree when to return.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

Keep the first change small enough to understand.

Use this only if a spike would answer your question. In this fictional proposal, one engineer and an experienced reviewer would explore a small isolated task with invented records. It has not been approved or run.

Name the question and the limits

Can an engineer use an approved coding agent on one small change, explain the result, and correct a deliberately introduced bug with the agreed support? Work in a sandbox. No real dataset, production credentials, deployments, or cross-service orchestration. Choose the tool with the people responsible for its use.

Agree the investment

Propose a cap of twelve total team hours over one week, including pairing, review, and fixes, plus $100 for tool use. Decide which work waits. Stop at the cap or if the work needs permissions outside the agreement. The reviewer must have time reserved before the spike starts.

Agree what you will check

Check behavior, tests, code explanation, debugging, and review effort. Compare with a similar task done without an agent if effort is the question; record scope and experience differences. The access design for the real app is a separate piece of work with its own owner.

A successful spike may justify another bounded step. It does not authorize a new team-wide workflow or release of the app.

Reading and reflection: see the companion at the end of this Guide >

A proposal you can review before starting.

Fictional and not yet agreed: explore one small development task while the real app's access design is investigated separately.

The question

Can an engineer complete, explain, and debug one small agent-assisted change with an experienced reviewer? Choose a task small enough to inspect fully.

The investment

Up to twelve total team hours over one week, including review and fixes, plus $100 in tool use. Reserve the reviewer and agree which work waits.

The boundary

Approved tool and sandbox. Invented records only. No production credentials, live data connections, deployment, or broader agent orchestration.

The check

Record behavior, understanding, debugging, and all review work. Stop at the cap or a permissions boundary. Review before extending the scope.

No result has been measured in this example. The plan helps discuss a possible investment; it is not evidence the approach works.

USE THESE QUESTIONS IN YOUR LAB >

Reading and reflection: see the companion at the end of this Guide >

Did it help, and could the team own it?

Record what happened before explaining it. Fast output is useful only in context: what was delivered, what people understood, what reviewers had to fix, and what remains to support.

Include the work around the code

Count setup, instructions, checking, review, corrections, and follow-up. Record tool costs and the experienced engineer's time. If a review queue grows, say so. Keep estimates of business value separate from measured benefits or time saved.

Look at understanding and quality

Could the engineer explain the change and find the bug? Which checks passed, failed, or were not performed? Keep the reviewer's notes. Passing tests for a sandbox task does not establish that the team can safely orchestrate a larger workflow.

Say what the evidence leaves open

One task with a strong reviewer tells you about that task under that support. It may not transfer to other people, repositories, or permissions. If the spike never ran, report the preparation findings. You have no productivity result to claim.

Try: "The first draft was quicker, but review took longer than we planned. We need to understand that before asking the whole team to work this way."

Reading and reflection: see the companion at the end of this Guide >

What can you responsibly recommend now?

Your PM needs something they can act on. Explain what you support, what needs to change, and what decision or help you need from someone else.

Be precise about the scope

You might recommend continuing with assistance on a particular task, revising the approach, preparing first, or stopping this use. Say which request that applies to. “Not this workflow with this team today” is a more useful conclusion than a verdict on all AI.

Name the work and decision owners

Who resolves access, provides review capacity, prioritizes the work, accepts security risk, and approves release? Those responsibilities may sit with different people. A PM and lead agreeing to explore something does not replace the organization’s required approvals.

If you still disagree

Write both positions accurately. Ask the relevant decision owner for a specific call, with the options and consequences. Say what the team will do while waiting and when you will return. Do not treat silence as agreement or let a proposed spike become a delivery promise.

Try: “I recommend preparing first. We need an access owner and review capacity. If we can resolve those, I can support this smaller next step. Until then, I’m not committing the team to ship it.”

Reading and reflection: see the companion at the end of this Guide >

Take a clear position into the conversation.

Use the parts you need now. You can prepare for a useful conversation without filling out an experiment plan or writing an essay.

Before you talk

The first Lab step helps you sort out your own reaction, observations, and questions. These private notes are excluded from saved drafts and shared outputs. Keep sensitive workplace information out of the examples you share.

Conversation Notes

Prepare a short account of the request, what your PM needs, your technical concerns, the options, and your recommendation. Edit it after the conversation. Distinguish your proposal from something everyone has agreed.

A spike and a return, when useful

Use the optional plan and results record if there is a question worth testing. Otherwise go straight to the recommendation. Come back after the next action: what changed, what did you learn about the work and each other, and what would help the team next time?

The aim is a conversation where your expertise helps shape the decision and your PM's context helps shape your recommendation.

Reading and reflection: see the companion at the end of this Guide >

What will be different next time?

The way you handle this request will also shape how the team works and how you and your PM talk.

Your judgment

What did you learn? Which concern became more specific? Did anything change your recommendation?

Your relationship

What do you understand about your PM's pressure now? What do they understand about engineering's responsibility?

The team's growth

Did people get time to understand the work? Who still needs help, and who has capacity to provide it?

The working conditions

What access, review, learning, or decision-making support should be in place before the next request arrives?

Choose a source that helps with a live question. Use the evidence with your own team and environment in mind.

AI assistance and learning coding skills >

App Service: authentication and authorization >

Choosing work for a coding agent >

OPEN THE TOOLS & STUDY LIBRARY > | SOURCES REVIEWED 23 OCT 2026

Reading and reflection: see the companion at the end of this Guide >

The thinking behind the tool.

Holistic Shaping asks what work is doing to people, as well as what people are doing at work. These short notes connect the Guide to Nic's teaching and a few practical sources. Read what helps with the situation in front of you. Links open the original resources.

Back in the Guide

CHAPTER 01

Make room for both contributions.

Your PM may see an opportunity you cannot yet see. You may see a failure mode they have not considered. Let each contribution change the discussion. You can hear the opportunity clearly and still recommend against the proposed approach.

Worth asking: What does each of us know that the other needs to hear?

Stop Chasing Psychological Safety
Nic Payne · Article

CHAPTER 04

Notice the story you have already written.

It is easy to hear another AI request and decide your PM only cares about a demo. Your concerns can be valid while that story is incomplete. Ask about their pressure, and explain yours without reducing either person to a role.

Worth asking: What have I assumed about my PM that I could ask them instead?

ChapGPT schools me on leading well
Nic Payne · Article

CHAPTER 05

The work is teaching people something.

A team learns from what it repeatedly does. If people only accept generated changes, they may miss opportunities to understand and debug them. If a more experienced person takes over every hard part, that has a cost too. Make time for people to work through difficulty with help.

Worth asking: What will this workflow give people a chance to practice?

When Mattering Isn't Enough
Nic Payne · Article

CHAPTER 08

Let their answer affect your recommendation.

Curiosity is useful when you are prepared to learn something. Bring a clear position and ask where it misses the need. You might keep the same security boundary and still find a better way to meet the request.

Worth asking: What answer from my PM would change the next move I propose?

The Search For Unapologetic Impracticality In Leadership: Part II - The Seduction of Certainty
Nic Payne · Article

CHAPTER 10

Look at what the measure leaves out.

A faster first draft can hide the hours someone else spends reviewing it. Count that work and notice who is carrying it. Include understanding, maintainability, and people's ability to recover from mistakes in the discussion of whether the change helped.

Worth asking: Who does more work under this approach, and have we counted it?

The Search For Unapologetic Impracticality In Leadership: Part III - How We Trained Ourselves to Mistake Certainty for Wisdom
Nic Payne · Article

CHAPTER 12

Change the conditions for the next request.

A useful conversation may reveal a missing access owner, a review bottleneck, or no time for learning. Those conditions will shape the next request too. Work with your PM and the people responsible to improve one of them, then return to see what changed.

Worth asking: What support could we put in place so this does not depend on one person holding everything together?

The Magic of Mr. Greenleaf — A Servant Leader's Guide to the Impossible
Nic Payne · Article

Explore more of Nic's Holistic Shaping work at Rebel Leaders.

The notes apply the teaching to these situations. They are not quotations unless marked.